

Formulario Big Data

HDFS

HDFS (Hadoop Distributed File System) è un file system distribuito, scalabile e tollerante ai guasti, progettato per archiviare file di grandi dimensioni su un cluster di hardware comune.

- **Blocchi:** I file in HDFS sono suddivisi in blocchi di grandi dimensioni (tipicamente 128MB o 256MB). Ogni blocco è memorizzato in modo indipendente.
- **Replicazione:** Ogni blocco è replicato su più nodi (il default è 3) per garantire la tolleranza ai guasti. Le repliche sono memorizzate su rack diversi per evitare la perdita di dati a causa di un guasto del rack.
- **NameNode:** Il server master che gestisce lo spazio dei nomi del file system e regola l'accesso ai file da parte dei client. Memorizza i metadati dei file (ad es. quali blocchi compongono un file e su quali DataNode sono memorizzati tali blocchi).
- **DataNodes:** I nodi slave che memorizzano i blocchi di dati effettivi. Sono responsabili della gestione delle richieste di lettura e scrittura da parte dei client.
- **Write-once-read-many:** HDFS è ottimizzato per letture di dati di grandi dimensioni e in streaming. I file vengono in genere scritti una volta e letti molte volte. Sono possibili accodamenti, ma non modifiche sul posto.

Teoria MapReduce

MapReduce è un modello di programmazione per l'elaborazione di grandi insiemi di dati con un algoritmo parallelo e distribuito su un cluster.

- **Fase Map:** I dati di input sono suddivisi in blocchi indipendenti che vengono elaborati dalle attività di map in modo completamente parallelo. La funzione map accetta una coppia chiave-valore ed emette un insieme di coppie chiave-valore intermedie.

- **Fase Reduce:** La funzione reduce viene chiamata per ogni chiave univoca e la sua lista di valori associati. Elabora la lista di valori ed emette un nuovo insieme di coppie chiave-valore, che sono l'output finale.
- **Località dei Dati:** Hadoop sposta il calcolo (codice) sui dati, non viceversa. Ciò minimizza il traffico di rete ed è un principio chiave per le prestazioni.
- **Shuffle and Sort:** Il framework mescola e ordina le coppie chiave-valore intermedie provenienti dai mapper. Tutte le coppie con la stessa chiave vengono raggruppate.

Implementazione Hadoop MapReduce

Esecuzione di un Job Hadoop MapReduce

- **InputSplit:** Un 'InputSplit' è un pezzo dell'input che viene elaborato da una singola attività di map. Per impostazione predefinita, l'InputFormat crea split della dimensione di un blocco HDFS.
- **Numero di Mapper:** Il numero di attività di map è determinato dal numero di 'InputSplit'. Se si dispone di un file da 1 GB archiviato in HDFS con una dimensione del blocco di 128 MB, si avranno 8 blocchi e quindi 8 'InputSplit', risultando in 8 attività di map.
- **Numero di Reducer:** Il numero di attività di reduce è configurato dallo sviluppatore nel codice del driver utilizzando `job.setNumReduceTasks(N)`. Se N è 0, non verranno eseguiti reducer (job solo map).
- **File di Output:** Il numero di file di output è uguale al numero di reducer. Ogni reducer scrive il proprio output in un file separato nella directory di output. Per i job solo map, ogni mapper scrive il proprio output in un file separato.

Frequenza delle Chiamate di Funzione

- **setup():** Chiamato **una volta** per attività (sia per i mapper che per i reducer) prima di qualsiasi altro metodo. Utilizzato per l'inizializzazione.
- **map():** Chiamato **una volta per ogni coppia chiave-valore** nell'InputSplit assegnato al mapper.
- **reduce():** Chiamato **una volta per ogni chiave univoca** e la sua lista di valori associati.
- **cleanup():** Chiamato **una volta** per attività (sia per i mapper che per i reducer) dopo l'esecuzione di tutti gli altri metodi. Utilizzato per la finalizzazione.

Combiner

Combiner (Mini-Reducer)

- Un Combiner è un'ottimizzazione opzionale che esegue una riduzione locale lato map. Prende l'output di un'attività di map e lo aggrega prima di inviarlo attraverso la rete ai reducer.
- Ciò riduce la quantità di dati trasferiti sulla rete.
- La classe Combiner è solitamente la stessa della classe Reducer, ma non è un requisito. `job.setCombinerClass(MyReducer.class)`
- **L'esecuzione non è garantita.** Hadoop può eseguire o meno il combiner. Pertanto, la logica di un job MapReduce non dovrebbe dipendere dall'esecuzione del combiner.
- Affinché un combiner possa essere utilizzato, la funzione di reduce deve essere associativa e commutativa.

Configurazione Job (II)

Configurazione del Job (Driver): L'oggetto 'Job' viene utilizzato nel driver per configurare il job MapReduce.

- **setJarByClass(class):** Imposta il Jar trovando da dove proviene una data classe.
- **setJobName(name):** Imposta il nome del job.
- **setMapperClass(class):** Imposta la classe 'Mapper'.
- **setReducerClass(class):** Imposta la classe 'Reducer'.
- **setCombinerClass(class):** Imposta la classe 'Combiner'.
- **setNumReduceTasks(num):** Imposta il numero di attività di reduce.
- **setInputFormatClass(class):** Imposta l'InputFormat per il job.
- **setOutputFormatClass(class):** Imposta l'OutputFormat per il job.
- **setMapOutputKeyClass(class):** Imposta la classe della chiave per l'output della map.
- **setMapOutputValueClass(class):** Imposta la classe del valore per l'output della map.
- **setOutputKeyClass(class):** Imposta la classe della chiave per l'output finale.
- **setOutputValueClass(class):** Imposta la classe del valore per l'output finale.

Tipi di Dati Hadoop

Tipi di Dati Hadoop: Hadoop utilizza i propri tipi di dati per la serializzazione.

- **Writable:** Interfaccia per oggetti che possono essere serializzati. Richiede l'implementazione di `write(DataOutput)` e `readFields(DataInput)`.

Tipi di Dati Hadoop

- **WritableComparable:** Interfaccia per oggetti che sono sia scrivibili che confrontabili. Le chiavi devono implementare questa interfaccia. Richiede l'implementazione di `compareTo(Object)`.
- **Tipi Comuni:**
 - **Text:** per le stringhe.
 - **IntWritable:** per gli interi.
 - **LongWritable:** per gli interi lunghi.
 - **FloatWritable:** per i valori float.
 - **DoubleWritable:** per i valori double.
 - **NullWritable:** un singleton per i valori nulli.

Formati di Input/Output

- **TextInputFormat:** Predefinito. La chiave è l'offset in byte della riga (LongWritable), il valore è il contenuto della riga (Text).
- **KeyValueTextInputFormat:** Divide le righe in base a un separatore (tabulazione per impostazione predefinita). La chiave è il testo prima del separatore, il valore è il testo dopo.
- **SequenceFileInputFormat:** Un formato di input per file di sequenza binari.
- **TextOutputFormat:** Predefinito. Scrive copie chiave-valore su una riga, separate da una tabulazione.

Context, Config & Contatori

1. Impostazione dei Parametri (nel Driver)

- **conf.set()**
 - **Prototipo:** `void set(String name, String value)`
 - **Descrizione:** Imposta un parametro personalizzato nell'oggetto 'Configuration' del job. Questo viene fatto nel Driver.
 - **Esempio:** `conf.set("my.parameter", "my.value");`

2. Accesso ai Parametri (nel Mapper/Reducer)

- **context.getConfiguration().get()**
 - **Prototipo:** `String get(String name)`
 - **Descrizione:** Recupera un valore di parametro impostato nel driver.
 - **Esempio:**
`String p =
context.getConfiguration()
.get("my.parameter");`

3. Definizione dei Contatori (nel Driver)

- I contatori sono definiti come un `enum` Java. Ciò fornisce un nome di gruppo (il nome dell'enum) e nomi di contatori (i valori dell'enum). Questo è tipicamente definito all'interno della classe Driver.
- **Esempio:** `public static enum MY_COUNTERS {
NUM_RECORDS, MALFORMED_RECORDS }
}`

4. Uso dei Contatori (nel Mapper/Reducer)

- **context.getCounter(...).increment()**
 - **Prototipo:** `void increment(long value)`
 - **Descrizione:** Recupera un oggetto 'Counter' e lo incrementa di un dato valore.
 - **Esempio:**
`context.getCounter(
MY_COUNTERS.NUM_RECORDS)
.increment(1);`

5. Emissione dell'Output (nel Mapper/Reducer)

- **context.write()**
 - **Prototipo:** `void write(KEYOUT key, VALUEOUT value)`
 - **Descrizione:** Emette una coppia chiave-valore dall'attività.
 - **Esempio:** `context.write(word, one);`

Pattern di Riepilogo

Pattern di Riepilogo sono usati quando è necessario aggregare i dati.

- **Conteggio/Somma:** Il classico WordCount è un esempio. I mapper emettono una chiave e un numero (ad es. 1), e i reducer sommano i numeri per ogni chiave.
- **Min/Max:** I mapper emettono una chiave e il valore numerico. I reducer scorrono i valori per una chiave e trovano il minimo o il massimo. Il pattern di combinazione in-mapper è molto efficace qui.
- **Media:** Richiede di tenere traccia sia di una somma che di un conteggio.
 - **Mapper:** Emette (chiave, (somma, conteggio)).
 - **Combiner:** Combina le coppie localmente, sommando sia le somme che i conteggi. Emette (chiave, (somma_parziale, conteggio_parziale)).
 - **Reducer:** Riceve (chiave, [(s1, c1), (s2, c2), ...]) e calcola la media finale come (somma_totale / conteggio_totale).

Pattern di Filtraggio

Pattern di Filtraggio: Utilizzato per selezionare un sottoinsieme dei dati.

- Questo è un **job solo map**.
- La funzione mapper ispeziona ogni record e decide se mantenerlo o scartarlo.
- Se il record soddisfa i criteri, il mapper lo scrive nell'output. Altrimenti, non fa nulla.
- Il numero di reducer è impostato su 0.
`job.setNumReduceTasks(0);`

Pattern di Ordinamento

Pattern di Ordinamento: MapReduce ordina automaticamente per chiave tra le fasi di map e reduce. Per un ordinamento più complesso, è necessario un **ordinamento secondario**.

- Per ordinare per valore (o parte della chiave), si crea una chiave composita, ad es. (chiave_primaria, chiave_secondaria).
- **Partitioner Personalizzato:** Assicura che tutte le chiavi con la stessa `chiave_primaria` vadano allo stesso reducer. Partiziona basandosi solo sulla parte `chiave_primaria` della chiave composita.
- **Comparatore di Raggruppamento Personalizzato:** Dice al reducer di raggruppare i record basandosi solo sulla `chiave_primaria`. In questo modo, il reducer riceve tutti i valori per quella `chiave_primaria` in una sola chiamata, con i valori ordinati per `chiave_secondaria`.

Join Lato Reduce

Join Lato Reduce: Un modo comune per unire due set di dati.

- **Mapper:** Elabora entrambi i set di dati. Per ogni record, emette la chiave di join come chiave intermedia. Il valore è una coppia contenente un "tag" per identificare il set di dati di origine e il valore originale. Ad es. (chiave_join, ("data1", valore1)) e (chiave_join, ("data2", valore2)).
- **Reducer:** Riceve tutti i valori per una data chiave di join. Ispeziona i tag per separare i valori di ciascun set di dati ed esegue la logica di join (ad es. un prodotto cartesiano). Questo può essere lento se alcune chiavi hanno molti valori.

Join Lato Map

Join Lato Map (Join Replicato): Utilizzato quando un set di dati è abbastanza piccolo da entrare nella memoria di ogni mapper.

- Il set di dati piccolo viene caricato nella memoria di ogni mapper nel metodo `setup()`. Questo viene spesso fatto utilizzando la `DistributedCache`.
- **Mapper:** Elabora il set di dati grande. Per ogni record, cerca i dati corrispondenti dal set di dati piccolo nella sua mappa in memoria ed esegue il join. Il risultato viene emesso direttamente.
- Questo è un **job solo map** ed è molto più efficiente di un join lato reduce poiché evita lo shuffling di rete.

Partitioner Personalizzato

Partitioner Personalizzato: Controlla quale reducer riceve quali chiavi intermedie.

- Per impostazione predefinita, le chiavi vengono partizionate da `hash(key) % num_reducers`.
- Un partitioner personalizzato è una classe che estende `Partitioner` e implementa il metodo `getPartition(...)`.
- **getPartition(key, value, numPartitions):**
 - **Prototipo:** `int getPartition(K key, V value, int numPartitions)`
 - **Descrizione:** Restituisce il numero della partizione (cioè, l'indice del reducer) per la coppia chiave-valore data.
 - **Uso:** Essenziale per l'ordinamento secondario per garantire che tutte le parti di una chiave composita vadano allo stesso reducer.
 - **Imposta nel Driver:**
 - **Set** **in** **Driver:**
`job.setPartitionerClass(MyPartitioner.class);`

Teoria RDD

Teoria RDD

- **RDD (Resilient Distributed Dataset):** La struttura dati fondamentale di Spark. È una collezione immutabile e distribuita di oggetti.
- **Esecuzione Lenta:** Le trasformazioni sugli RDD non vengono calcolate immediatamente. Spark costruisce un **DAG (Directed Acyclic Graph)** di operazioni. I calcoli vengono eseguiti solo quando viene chiamata un'azione.
- **Trasformazioni:** Creano un nuovo RDD da uno esistente (ad es. `map`, `filter`). Sono pigre.
- **Azioni:** Attivano l'esecuzione del DAG e restituiscono un valore al programma driver o scrivono i dati su un sistema di archiviazione esterno (ad es. `count`, `collect`).

SparkContext

SparkContext

- **Creazione:**
 - `from pyspark import SparkConf, SparkContext`
 - `conf = SparkConf().setAppName(name)`
 - `sc = SparkContext(conf=conf)`
- **parallelize(list, numSlices):** Crea un RDD da una lista Python.
- **textFile(path, minPartitions):** Legge un file di testo e lo restituisce come un RDD di stringhe.
- **stop():** Arresta lo SparkContext.

Trasformazioni RDD (I)

Trasformazioni RDD (Base)

- **map(f):** `map(f: funzione) -> RDD`
 - Restituisce un nuovo RDD applicando una funzione a ogni elemento.
 - `rdd.map(lambda x: x + 1)`

- **flatMap(f):** `flatMap(f: funzione) -> RDD`
 - Simile a `map`, ma ogni elemento di input può essere mappato a 0 o più elementi di output.
 - `rdd.flatMap(lambda s: s.split(' '))`
- **filter(f):** `filter(f: funzione) -> RDD`
 - Restituisce un nuovo RDD contenente solo gli elementi che soddisfano una funzione.
 - `rdd.filter(lambda x: x > 10)`
- **distinct():** `distinct() -> RDD`
 - Restituisce un nuovo RDD contenente gli elementi distinti.
 - `rdd.distinct()`
- **sample(withReplacement, fraction, seed):** `sample(...) -> RDD`
 - Restituisce un sottoinsieme campionato di questo RDD.
 - `rdd.sample(False, 0.5)`

Trasformazioni RDD (II)

Trasformazioni RDD (Pair RDD)

- **reduceByKey(func):** `reduceByKey(func) -> RDD`
 - Unisce i valori per ogni chiave usando una funzione di riduzione associativa e commutativa.
 - `rdd.reduceByKey(lambda a, b: a + b)`
- **groupByKey():** `groupByKey() -> RDD`
 - Raggruppa i valori per ogni chiave in un RDD in una singola sequenza.
 - `rdd.groupByKey()`

- **sortByKey(ascending):** `sortByKey(asc) -> RDD`
 - Ordina questo RDD, che si presume consista di coppie (chiave, valore).
 - `rdd.sortByKey()`
- **join(other):** `join(other) -> RDD`
 - Restituisce un RDD contenente tutte le coppie di elementi con chiavi corrispondenti in `self` e `other`.
 - `rdd1.join(rdd2)`
- **leftOuterJoin(other):** `leftOuterJoin(other) -> RDD`
 - Esegue un left outer join di `self` e `other`.
 - `rdd1.leftOuterJoin(rdd2)`
- **cogroup(other):** `cogroup(other) -> RDD`
 - Per ogni chiave `k` in `self` o `other`, restituisce un RDD risultante che contiene una tupla con la lista di valori per quella chiave in `self` e `other`.
 - `rdd1.cogroup(rdd2)`
- **rightOuterJoin(other):** `rightOuterJoin(other) -> RDD`
 - Esegue un right outer join di `self` e `other`.
 - `rdd1.rightOuterJoin(rdd2)`

Azioni RDD (I)

- **collect():** `collect() -> list`
 - Restituisce una lista che contiene tutti gli elementi di questo RDD.
 - `rdd.collect()`
- **count():** `count() -> int`
 - Restituisce il numero di elementi in questo RDD.
 - `rdd.count()`

Azioni RDD (II)

- **take(n):** take(n) -> list
 - Prende i primi n elementi dell’RDD.
 - rdd.take(5)
- **first():** first() -> T
 - Restituisce il primo elemento di questo RDD.
 - rdd.first()
- **reduce(f):** reduce(f) -> T
 - Riduce gli elementi di questo RDD utilizzando l’operatore binario commutativo e associativo specificato.
 - rdd.reduce(lambda a, b: a + b)
- **saveAsTextFile(path):** saveAsTextFile(path)
 - Salva questo RDD come file di testo, usando le rappresentazioni di stringa degli elementi.
 - rdd.saveAsTextFile("output")
- **countByKey():** countByKey() -> dict
 - Conta il numero di elementi per ogni chiave e restituisce il risultato al master come dizionario.
 - rdd.countByKey()
- **countByValue():** countByValue() -> dict
 - Restituisce il conteggio di ogni valore univoco in questo RDD come un dizionario di coppie (valore, conteggio).
 - rdd.countByValue()
- **max():** max() -> T
 - Restituisce il massimo di questo RDD.
 - rdd.max()

- **min():** min() -> T
 - Restituisce il minimo di questo RDD.
 - rdd.min()
- **sum():** sum() -> number
 - Somma gli elementi di questo RDD.
 - rdd.sum()

Teoria PySpark SQL

PySpark SQL è un modulo Spark per l’elaborazione di dati strutturati. Fornisce un’astrazione di programmazione chiamata **DataFrame**.

- I **DataFrame** sono collezioni distribuite di dati organizzati in colonne con nome, simili a una tabella in un database relazionale.
- Spark SQL utilizza un ottimizzatore chiamato **Catalyst**, che rende i programmi **DataFrame** più veloci di quelli basati su RDD.

Descrizioni Classi

SparkSession: Il punto di ingresso per programmare Spark con l’API Dataset e DataFrame. Viene utilizzato per creare **DataFrame**, registrare **DataFrame** come tabelle, eseguire SQL su tabelle, memorizzare nella cache le tabelle e leggere file parquet.

DataFrame: Una collezione distribuita di dati organizzati in colonne con nome. È concettualmente equivalente a una tabella in un database relazionale o a un data frame in R/Python, ma con ottimizzazioni più ricche sotto il cofano.

Column: Un’espressione di colonna in un **DataFrame**. Le istanze di **Column** vengono create selezionando una colonna da un **DataFrame**.

GroupedData: Metodi di aggregazione, restituiti da **DataFrame.groupBy()**. Utilizzato per eseguire operazioni di aggregazione sui dati raggruppati.

createDataFrame: createDataFrame(data, schema) -> DataFrame
Crea un **DataFrame** da un RDD, una lista o un pandas.DataFrame.
spark.createDataFrame([(1, 'a'), (2, 'b')], ['id', 'val'])

read: read() -> DataFrameReader
Restituisce un **DataFrameReader** che può essere utilizzato per leggere i dati come DataFrame.
spark.read.format("csv").load("path")

sql: sql(sqlQuery) -> DataFrame
Esegue una query SQL e restituisce il risultato come DataFrame.
spark.sql("SELECT * FROM table")

stop: stop() -> None
Arresta la **SparkSession**.
spark.stop()

conf: conf() -> SparkConf
Restituisce l’oggetto **SparkConf** di questa sessione.
spark.conf.get("spark.sql.shuffle.partitions")

range: range(start, end, step, numPartitions) -> DataFrame
Crea un **DataFrame** con una singola colonna denominata id, contenente elementi in un intervallo.
spark.range(1, 7, 2).show()

udf: udf() -> UDFRegistration
Restituisce un oggetto **UDFRegistration** per la registrazione delle UDF.
spark.udf.register("my_udf", lambda x: x * 2)

Funzioni DataFrame (I)

agg: agg(*exprs) -> DataFrame
Calcola aggregati e restituisce un **DataFrame**.
df.groupBy("age").agg("*: count")

Funzioni DataFrame (II)

alias: `alias(alias) -> DataFrame`

Restituisce un nuovo DataFrame con un alias impostato.

`df.alias("my_df")`

cache: `cache() -> DataFrame`

Mantiene il DataFrame con il livello di archiviazione predefinito (MEMORY_AND_DISK).

`df.cache()`

coalesce: `coalesce(numPartitions) -> DataFrame`

Restituisce un nuovo DataFrame con esattamente numPartitions partizioni.

`df.coalesce(1)`

collect: `collect() -> list[Row]`

Restituisce tutti i record come una lista di Row.

`df.collect()`

columns: `columns -> list[str]`

Restituisce tutti i nomi delle colonne come una lista di stringhe.

`df.columns`

count: `count() -> int`

Restituisce il numero di righe nel DataFrame.

`df.count()`

createOrReplaceTempView: `cOTV(name) -> None`

Crea o sostituisce una vista temporanea locale con questo DataFrame.

`df.createOrReplaceTempView("people")`

distinct: `distinct() -> DataFrame`

Restituisce un nuovo DataFrame contenente le righe distinte.

`df.distinct()`

drop: `drop(*cols) -> DataFrame`

Restituisce un nuovo DataFrame che elimina le colonne specificate.

`df.drop('age', 'name')`

crossJoin: `crossJoin(other) -> DataFrame`

Restituisce il prodotto cartesiano con un altro DataFrame.

`df1.crossJoin(df2)`

dropDuplicates: `dropDuplicates(subset) -> DataFrame`

Restituisce un nuovo DataFrame con le righe duplicate rimosse, opzionalmente considerando solo alcune colonne.

`df.dropDuplicates(["name", "department"])`

filter: `filter(condition) -> DataFrame`

Filtra le righe usando la condizione data.

`df.filter(df['age'] > 21)`

first: `first() -> Row`

Restituisce la prima riga come Row.

`df.first()`

groupBy: `groupBy(*cols) -> GroupedData`

Raggruppa il DataFrame usando le colonne specificate.

`df.groupBy("department")`

join: `join(other, on, how) -> DataFrame`

Si unisce con un altro DataFrame.

`df1.join(df2, df1.id == df2.id, 'outer')`

limit: `limit(n) -> DataFrame`

Restituisce un nuovo DataFrame prendendo le prime n righe.

`df.limit(5)`

orderBy: `orderBy(*cols, ascending) -> DataFrame`

Restituisce un nuovo DataFrame ordinato per la colonna o le colonne specificate.

`df.orderBy("age", ascending=False)`

persist: `persist() -> DataFrame`

Mantiene questo DataFrame con il livello di archiviazione predefinito.

`df.persist()`

rdd: `rdd -> RDD[Row]`

Restituisce il contenuto come un RDD di Row.

`df.rdd`

select: `select(*cols) -> DataFrame`

Proietta un insieme di espressioni e restituisce un nuovo DataFrame.

`df.select("name", "age")`

show: `show(n, truncate, vertical) -> None`

Stampa le prime n righe sulla console.

`df.show(5)`

sort: `sort(*cols, ascending) -> DataFrame`

Restituisce un nuovo DataFrame ordinato per la colonna o le colonne specificate.

`df.sort(df.age.desc())`

union: `union(other) -> DataFrame`

Restituisce un nuovo DataFrame contenente l'unione delle righe di questo e un altro frame.

`df1.union(df2)`

where: `where(condition) -> DataFrame`

Un alias per filter().

`df.where(df.age > 21)`

withColumn: `withColumn(colName, col) -> DataFrame`

Restituisce un nuovo DataFrame aggiungendo una colonna o sostituendone una esistente.

`df.withColumn("age_plus_1", df.age + 1)`

withColumnRenamed: `withColumnRenamed(old, new) -> DataFrame`

Restituisce un nuovo DataFrame rinominando una colonna esistente.

`df.withColumnRenamed("age", "user_age")`

summary: `summary(statistics) -> DataFrame`

Calcola le statistiche specificate per le colonne numeriche e di stringa.

`df.summary("count", "min", "max").show()`

write: `write() -> DataFrameWriter`

Interfaccia per salvare il contenuto del DataFrame non in streaming in un archivio esterno.

`df.write.format("parquet").save("path")`

Funzioni Column

alias: alias(alias) -> Column

Dà alla colonna un alias.

```
df.age.alias("user_age")
```

asc: asc() -> Column

Restituisce un'espressione di ordinamento basata sull'ordine ascendente della colonna.

```
df.sort(df.age.asc())
```

between: between(lower, upper) -> Column

Un'espressione booleana che viene valutata come vera se il valore di questa espressione è compreso tra i limiti dati.

```
df.select(df.name, df.age.between(18, 30))
```

cast: cast(dataType) -> Column

Converte la colonna in un tipo di dati diverso.

```
df.age.cast("string")
```

contains: contains(other) -> Column

Un'espressione booleana che viene valutata come vera se il valore di questa espressione contiene il valore dato.

```
df.filter(df.name.contains("o"))
```

desc: desc() -> Column

Restituisce un'espressione di ordinamento basata sull'ordine discendente della colonna.

```
df.sort(df.age.desc())
```

endswith: endswith(other) -> Column

La stringa termina con un'altra stringa.

```
df.filter(df.name.endswith("n"))
```

isNull: isNull() -> Column

Vero se l'espressione corrente è nulla.

```
df.filter(df.age.isNull())
```

isNotNull: isNotNull() -> Column

Vero se l'espressione corrente NON è nulla.

```
df.filter(df.age.isNotNull())
```

like: like(other) -> Column

Espressione SQL like.

```
df.filter(df.name.like("A%"))
```

startswith: startswith(other) -> Column

La stringa inizia con un'altra stringa.

```
df.filter(df.name.startswith("A"))
```

Funzioni GroupedData

agg: agg(*exprs) -> DataFrame

Calcola aggregati e restituisce un DataFrame.

```
df.groupBy("dep").agg("salary": "avg")
```

avg: avg(*cols) -> DataFrame

Calcola la media per le colonne numeriche per ogni gruppo.

```
df.groupBy("dep").avg("salary")
```

count: count() -> DataFrame

Conta il numero di record per ogni gruppo.

```
df.groupBy("dep").count()
```

max: max(*cols) -> DataFrame

Calcola il massimo per le colonne numeriche per ogni gruppo.

```
df.groupBy("dep").max("salary")
```

sum: sum(*cols) -> DataFrame

Calcola la somma per le colonne numeriche per ogni gruppo.

```
df.groupBy("dep").sum("salary")
```

mean: mean(*cols) -> DataFrame

Calcola la media per le colonne numeriche per ogni gruppo.

```
df.groupBy("dep").mean("salary")
```

min: min(*cols) -> DataFrame

Calcola il minimo per le colonne numeriche per ogni gruppo.

```
df.groupBy("dep").min("salary")
```

pyspark.sql.functions (I)

abs: abs(col) -> Column

Calcola il valore assoluto.

```
from pyspark.sql.functions import abs
df.select(abs(df.value))
```

avg: avg(col) -> Column

Restituisce la media dei valori in un gruppo.

```
from pyspark.sql.functions import avg
df.select(avg(df.age))
```

coalesce: coalesce(*cols) -> Column

Restituisce la prima colonna che non è nulla.

```
from pyspark.sql.functions import coalesce
df.select(coalesce(df.col1, df.col2))
```

col: col(col) -> Column

Restituisce una colonna basata sul nome della colonna dato.

```
from pyspark.sql.functions import col
df.select(col("age"))
```

concat: concat(*cols) -> Column

Concatena più colonne di stringhe di input in una singola colonna di stringhe.

```
from pyspark.sql.functions import concat
df.select(concat(df.first, lit(' '),
df.last))
```

count: count(col) -> Column

Restituisce il numero di elementi in un gruppo.

```
from pyspark.sql.functions import count
df.select(count(df.age))
```

dayofmonth: dayofmonth(col) -> Column

Estrae il giorno del mese come intero da una data/timestamp data.

```
from pyspark.sql.functions import dayofmonth
df.select(dayofmonth(df.date))
```

lit: lit(col) -> Column

Crea una colonna di valore letterale.

```
from pyspark.sql.functions import lit
df.withColumn("one", lit(1))
```

sum: sum(col) -> Column

Restituisce la somma dei valori in un gruppo.

```
from pyspark.sql.functions import sum
df.select(sum(df.salary))
```

pyspark.sql.functions (II)

upper: upper(col) -> Column

Converte un'espressione di stringa in maiuscolo.

```
from pyspark.sql.functions import upper
df.select(upper(df.name))
```

when: when(condition, value) -> Column

Valuta una lista di condizioni e restituisce una delle molteplici espressioni di risultato possibili.

```
from pyspark.sql.functions import when
df.select(when(df.age > 20, 1).otherwise(0))
```

udf: udf(f, returnType) -> UserDefinedFunction

Crea una funzione definita dall'utente (UDF).

```
from pyspark.sql.functions import udf
from pyspark.sql.types import IntegerType
slen = udf(lambda s: len(s), IntegerType())
df.select(slen(df.name))
```

aggregate: aggregate(col, initialValue, merge, finish) -> Column

Aggrega su una lista di valori.

```
from pyspark.sql.functions import aggregate
```

dense_rank: dense_rank() -> Column

Funzione finestra che restituisce il rank delle righe all'interno di una partizione di finestra, senza interruzioni.

```
from pyspark.sql.functions import dense_rank
```

split: split(str, pattern) -> Column

Divide la stringa str intorno alle occorrenze che corrispondono al pattern.

```
from pyspark.sql.functions import split
```

substring: substring(str, pos, len) -> Column

La sottostringa inizia alla posizione pos ed è di lunghezza len quando str è di tipo String.

```
from pyspark.sql.functions import substring
```

trim: trim(col) -> Column

Elimina gli spazi da entrambe le estremità per la colonna di stringa specificata.

```
from pyspark.sql.functions import trim
```

Teoria PySpark MLlib

pyspark.ml è la nuova API basata su DataFrame per MLlib. È l'API raccomandata.

- **Transformer:** Un algoritmo che può trasformare un DataFrame in un altro DataFrame. Ad es. un trasformatore di feature o un modello addestrato. Ha un metodo `.transform()`.
- **Estimator:** Un algoritmo che può essere adattato a un DataFrame per produrre un Transformer. Ad es. un algoritmo di apprendimento come `LogisticRegression`. Ha un metodo `.fit()`.
- **Pipeline:** Concatena più Transformer ed Estimator per specificare un flusso di lavoro di ML. Le fasi vengono eseguite in ordine e il DataFrame di input viene trasformato man mano che passa attraverso ogni fase.
- **Parameter:** Tutti i Transformer e gli Estimator condividono un'API comune per la specifica dei parametri.

Estrazione Feature MLlib (I)

VectorAssembler: VectorAssembler(inputCols, outputCol) -> Transformer

Un trasformatore di feature che unisce più colonne in una singola colonna vettoriale.

```
assembler = VectorAssembler(
    inputCols=["c1", "c2"],
    outputCol="features")
```

StandardScaler: StandardScaler(inputCol, outputCol) -> Estimator

Un estimator che scala una colonna vettoriale di feature per avere media zero e varianza unitaria.

```
scaler = StandardScaler(
    inputCol="features",
    outputCol="scaledFeatures")
```

StringIndexer: StringIndexer(inputCol, outputCol) -> Estimator

Un estimator che codifica una colonna di stringhe di etichette in una colonna di indici di etichette.

```
indexer = StringIndexer(
    inputCol="category",
    outputCol="categoryIndex")
```

IndexToString: IndexToString(inputCol, outputCol) -> Transformer

Un trasformatore che mappa una colonna di indici di nuovo a una colonna di valori di stringa originali.

```
converter = IndexToString(
    inputCol="prediction",
    outputCol="originalCategory")
```

Tokenizer: Tokenizer(inputCol, outputCol) -> Transformer

Un trasformatore che converte la stringa di input in minuscolo e poi la divide per spazi bianchi.

```
tokenizer = Tokenizer(
    inputCol="text", outputCol="words")
```

StopWordsRemover:

StopWordsRemover(inputCol, outputCol) -> Transformer

Un trasformatore che filtra le stop word dall'input.

```
remover = StopWordsRemover(
    inputCol="words",
    outputCol="filtered")
```

HashingTF: HashingTF(inputCol, outputCol, numFeatures) -> Transformer

Un trasformatore che mappa una sequenza di termini alle loro frequenze di termine usando l'hashing trick.

```
hashingTF = HashingTF(
    inputCol="filtered",
    outputCol="rawFeatures")
```

IDF: IDF(inputCol, outputCol) -> Estimator

Un estimator che calcola l'Inverse Document Frequency (IDF) e poi scala le frequenze dei termini.

```
idf = IDF(inputCol="rawFeatures",
    outputCol="features")
```

Classificazione & Clustering MLlib (I)

LogisticRegression: LogisticRegression(featuresCol, labelCol) -> Estimator

Un algoritmo di classificazione per problemi di classificazione binaria.

```
lr = LogisticRegression(
    featuresCol='features',
    labelCol='label')
```


Classificazione & Clustering MLlib (II)

DecisionTreeClassifier: DTC(featuresCol, labelCol) -> Estimator
Un algoritmo di classificazione che predice la classe di una variabile target apprendendo semplici regole di decisione.
dt = DecisionTreeClassifier()

KMeans: KMeans(featuresCol, k) -> Estimator
Un algoritmo di clustering che raggruppa i punti dati in un numero predefinito di cluster.
kmeans = KMeans(k=2)

Valutazione Modello MLlib

BinaryClassificationEvaluator:
BCE(rawPredictionCol, labelCol) -> Evaluator
Valutatore per la classificazione binaria, che si aspetta due colonne di input: rawPrediction e label.
evaluator = BinaryClassificationEvaluator()

MulticlassClassificationEvaluator:
MCE(predictionCol, labelCol) -> Evaluator
Valutatore per la classificazione multiclasse.
evaluator = MulticlassClassificationEvaluator(metricName="accuracy")

evaluate: evaluate(dataset) -> float
Valuta il modello su un set di dati e restituisce la metrica.
accuracy = evaluator.evaluate(predictions)

Ottimizzazione Parametri MLlib

Pipeline: Pipeline(stages) -> Estimator
Una pipeline concatena più estimator e transformer per specificare un flusso di lavoro di ML.
pipeline = Pipeline(stages=[s1, s2, s3])

ParamGridBuilder: ParamGridBuilder() -> ParamGridBuilder
Serve per costruire una griglia di parametri usata nella selezione basata sulla ricerca a griglia.
grid = ParamGridBuilder().build()
estimatorParamMaps=grid,
evaluator=eval, numFolds=3)

CrossValidator: CrossValidator(estimator, eParamMaps, evaluator) -> Estimator
Un estimator per la convalida incrociata k-fold.
cv = CrossValidator(estimator=pl,
estimatorParamMaps=grid,
evaluator=eval, numFolds=3)

Teoria Spark Streaming

Spark Streaming elabora un flusso di dati di input dal vivo dividendolo in piccoli batch.

- **DStream (Discretized Stream):** L'astrazione di base. È una sequenza di RDD, dove ogni RDD contiene dati da un certo intervallo di tempo.
- **Micro-batching:** Spark Streaming riceve i dati e li memorizza nel buffer in memoria. Quindi, il motore di Spark esegue un breve task (un micro-batch) sui nuovi dati.
- **StreamingContext:** Il punto di ingresso principale per tutte le funzionalità di streaming.

Contesto Spark Streaming

StreamingContext: StreamingContext(sc, batchDuration) -> ssc
Crea un nuovo StreamingContext.
from pyspark.streaming import StreamingContext
ssc = StreamingContext(sc, 10)

socketTextStream: socketTextStream(host, port) -> DStream
Crea un flusso di input da un socket TCP.
lines = ssc.socketTextStream("localhost", 9999)

start: start() -> None
Avvia l'esecuzione dei flussi.
ssc.start()

awaitTermination: awaitTermination() -> None
Attende l'arresto dell'esecuzione.
ssc.awaitTermination()

checkpoint: checkpoint(directory) -> None
Imposta la directory di checkpoint. Deve trovarsi su un file system tollerante ai guasti. Richiesto for trasformazioni stateful.
ssc.checkpoint("hdfs:///checkpoints")

Trasformazioni DStream

window: window(windowLength, slideInterval) -> DStream
Restituisce un nuovo DStream calcolato in base a batch finestrati.
windowedStream = ds.window(30, 10)

reduceByKeyAndWindow: rBKW(func, invFunc, wL, sI) -> DStream
Aggrega i valori di ogni chiave su una finestra scorrevole.
ds.reduceByKeyAndWindow(add, sub, 30, 10)

updateStateByKey: updateStateByKey(updateFunc) -> DStream
Consente di mantenere uno stato arbitrario aggiornandolo continuamente con nuove informazioni.
ds.updateStateByKey(updateFunction)

transform: transform(func) -> DStream
Applica una funzione RDD-to-RDD a ogni RDD del DStream. Consente operazioni RDD arbitrarie.
ds.transform(lambda rdd: rdd.sortByKey())

Azioni DStream

pprint: pprint() -> None
Stampa i primi 10 elementi di ogni batch di dati in un DStream sul driver.
counts.pprint()

saveAsTextFiles: saveAsTextFiles(prefix, [suffix]) -> None
Salva il contenuto del DStream come file di testo. Viene creata una nuova directory per ogni batch.
counts.saveAsTextFiles("output", "txt")

foreachRDD: foreachRDD(func) -> None
L'operatore di output più generico. Applica una funzione a ogni RDD nel DStream.
ds.foreachRDD(lambda rdd: rdd.toDF().write...)